Mock OA — Debrief and Spoilers

Read only after attempting

Prep materials — Anthropic Fellows OA

August 2026

Contents

SPOILERS — read only AFTER attempting the mock **1**
Built-in traps (deliberate, mirroring the real OA complaints) 1
How to grade yourself 1
Debrief questions 2
Reference solution 2

SPOILERS — read only AFTER attempting the mock

Built-in traps (deliberate, mirroring the real OA complaints)

1. **README vs. code mismatch**: the Level 1 README example shows a dict (`{1: 5, 2: 8}`) but `run_fifo`'s docstring — and the tests — expect a **list of tuples in completion order**. Lesson: when README and code disagree, trust tests > signatures/docstrings > README.
2. **Tick-order pedantry**: Level 2's free → admit → work order changes answers. A request finishing "at the end of tick t" frees memory at the start of tick t+1, so a waiting request admitted then starts prefill on tick t+1, not t.
3. **Prefill-completion tick generates no token** — classic off-by-one.
4. **Head-of-line blocking** in admission: a small request that fits may NOT skip a big one ahead of it in the queue.
5. **`Request.fresh()`** exists because runtime state is mutable — if you run a simulation twice on the same objects you get garbage. The engine file hints at this; noticing it is part of the exercise.
6. **Level 4 reverses a Level 2 rule**: head-of-line blocking is gone, and admission order changes. Real OAs do this — a later level invalidates an assumption you baked in. Write Level 2 so the admission policy is easy to swap, and re-read each level's spec instead of assuming continuity.
7. **Eviction determinism**: victim = lowest priority, ties by *highest* id (not lowest — read carefully). Evicted requests lose prefill progress and can't be re-admitted the same tick, which prevents livelock.

How to grade yourself

- `python tests.py` — your score.
- `python tests.py --solution` — verify the reference gets 800/800.

- 500+ in 90 minutes: strong. 300–500: fine — focus on whichever level broke. Levels 1–3 fully solved (600/800) is a solid outcome; level 4 is deliberately the time sink.

Debrief questions

- How long before you wrote your first line of code? (Target: ≤15 min of reading, including tests.)
- Did you read `tests.py` before implementing? You were allowed to. The expected values there resolve every ambiguity in the README.
- Which trap cost you the most time?

Reference solution

`solution/scheduler_solution.py`. To rerun the mock later, restore `scheduler.py` stubs and try again with different self-imposed time limits.